

Manual de Instalación y Uso

Clasificación de pacientes con múltiples neoplasias mediante procesamiento de lenguaje natural y modelos ligeros

Trabajo Fin de Grado

Autor: Juan Jesús Rus Muñoz

Tutor: José Manuel Jerez Aragonés

Cotutor: Francisco Javier Moreno Barea

Universidad: Universidad de Málaga

1. Introducción

El presente documento describe el procedimiento de instalación, configuración y uso de la aplicación web desarrollada para la clasificación automática de neoplasias a partir de historias clínicas electrónicas en formato texto. La aplicación ha sido implementada como una herramienta local basada en FastAPI y modelos de lenguaje de la familia transformers, permitiendo analizar texto clínico introducido manualmente o mediante la carga de archivos `.txt`.

El sistema proporciona dos modalidades principales de análisis. En primer lugar, incorpora una clasificación binaria orientada a determinar si una historia clínica corresponde a un caso con una única neoplasia o con múltiples neoplasias. En segundo lugar, dispone de una clasificación multietiqueta que permite identificar posibles localizaciones tumorales mediante un esquema de inferencia en cascada. Adicionalmente, la aplicación integra explicaciones mediante LIME para facilitar la interpretación de la predicción binaria obtenida por el modelo.

La aplicación está diseñada para ejecutarse en entorno local y para cargar los modelos previamente entrenados desde rutas configuradas en un archivo YAML. Este enfoque permite separar el código fuente de los artefactos de modelo, manteniendo una estructura clara entre la aplicación, la configuración y los recursos de inferencia.

Este manual está dirigido a usuarios técnicos que necesiten instalar, configurar y ejecutar la aplicación en un equipo con sistema operativo Windows, utilizando CPU como dispositivo de ejecución.

2. Requisitos de ejecución

2.1. Requisitos hardware

La aplicación no establece requisitos hardware específicos. Para su ejecución local únicamente se necesita un equipo convencional capaz de ejecutar Python, abrir un navegador web y almacenar los modelos entrenados indicados en la configuración.

No se requiere GPU. Todos los comandos y procedimientos descritos en este documento asumen ejecución en CPU.

2.2. Requisitos software

El entorno software necesario está formado por:

- Sistema operativo Windows.
- Python instalado y accesible desde terminal.
- `pip`, gestor de paquetes de Python.
- Código fuente del proyecto.
- Modelos entrenados descargados desde el enlace de Drive facilitado y ubicados en las rutas configuradas en `config_app.yaml`.

Las dependencias Python reales declaradas por la aplicación son las siguientes:

```
fastapi==0.125.0
starlette==0.50.0
uvicorn==0.38.0
pydantic==2.10.2
transformers==4.50.0
torch==2.4.1
pyyaml==6.0.2
jinja2==3.1.6
python-multipart==0.0.21
lime==0.2.0.1
sentencepiece
```

Estas dependencias se encuentran recogidas en el archivo:

```
app/requirements.txt
```

3. Organización de carpetas del proyecto

La estructura principal del proyecto es la siguiente:

```
NeoplasiaClassifier-ES/
|
|-- app/
|   |-- main.py
|   |-- inference.py
|   |-- lime_utils.py
|   |-- config_app.yaml
|   |-- requirements.txt
|   |-- static/
|   |   |-- styles.css
```

```
|  |-- templates/
|      |-- index.html
|
|-- models/
|      |-- README.md
|
|-- script/
|
|-- README.md
```

La carpeta `app/` contiene los componentes necesarios para ejecutar la interfaz web:

- `main.py`: define la aplicación FastAPI, las rutas web y la comunicación con la interfaz.
- `inference.py`: implementa la carga de modelos, el preprocesamiento del texto y la lógica de inferencia.
- `lime_utils.py`: contiene la integración con LIME para generar explicaciones en modo binario.
- `config_app.yaml`: centraliza las rutas de los modelos y parámetros de inferencia.
- `requirements.txt`: lista las dependencias necesarias para ejecutar la aplicación.
- `templates/`: contiene la plantilla HTML de la interfaz.
- `static/`: contiene los estilos CSS de la interfaz.

La carpeta `models/` se reserva para alojar localmente los modelos descargados. Los modelos no se incluyen directamente en el repositorio debido a su tamaño, por lo que deben obtenerse mediante el enlace de Drive facilitado junto con el proyecto. En la configuración actual, `app/config_app.yaml` utiliza rutas relativas desde la carpeta `app/` hacia `../models/`.

La carpeta `script/` agrupa utilidades auxiliares de entrenamiento, evaluación y análisis. No es necesaria para ejecutar la aplicación web de inferencia.

4. Instalación paso a paso

4.1. Abrir una terminal de Windows

Abra una terminal de PowerShell y sitúese en la carpeta del proyecto. Un ejemplo de comando en Windows es:

```
cd "C:\TFG\NeoplasiaClassifier-ES"
```

Desde esta ruta se encuentra disponible la carpeta `app/`, que contiene la aplicación web.

4.2. Comprobar la instalación de Python

Antes de crear el entorno virtual, verifique que Python está disponible:

```
python --version
```

También puede comprobar la disponibilidad de `pip` mediante:

```
python -m pip --version
```

Si ambos comandos devuelven información de versión, el entorno base de Python está accesible desde la terminal.

Si Python no está instalado o el comando anterior no se reconoce, debe instalarse Python antes de continuar. Para esta aplicación se recomienda utilizar **Python 3.12.6**.

La instalación puede realizarse desde la página oficial de Python:

```
https://www.python.org/downloads/release/python-3126/
```

Durante la instalación en Windows, marque la opción **Add python.exe to PATH** antes de pulsar **Install Now**. Esta opción permite ejecutar Python desde PowerShell sin configurar rutas manualmente.

Una vez finalizada la instalación, cierre y vuelva a abrir la terminal y compruebe de nuevo:

```
python --version  
python -m pip --version
```

La salida esperada debe mostrar una versión de Python 3.12, preferiblemente:

```
Python 3.12.6
```

4.3. Creación de un entorno virtual

Se recomienda utilizar un entorno virtual para aislar las dependencias de la aplicación respecto al resto del sistema. Desde la raíz del proyecto, ejecute:

```
python -m venv .venv
```

Este comando crea una carpeta `.venv/` con el entorno de Python asociado al proyecto.

4.4. Activación del entorno virtual

En PowerShell, active el entorno virtual con el siguiente comando:

```
.\.venv\Scripts\Activate.ps1
```

Tras la activación, la terminal mostrará el nombre del entorno al inicio de la línea de comandos, normalmente como `(.venv)`.

Si se utiliza la consola clásica de Windows, el comando equivalente es:

```
.venv\Scripts\activate.bat
```

4.5. Actualización de pip

Con el entorno virtual activado, se recomienda actualizar `pip`:

```
python -m pip install --upgrade pip
```

4.6. Instalación de dependencias

Las dependencias de ejecución de la aplicación se encuentran en `app/requirements.txt`. Desde la raíz del proyecto, instale los paquetes necesarios mediante:

```
python -m pip install -r app\requirements.txt
```

El archivo de dependencias fija versiones concretas para mejorar la reproducibilidad del entorno. Incluye FastAPI y Starlette para la aplicación web, Uvicorn para ejecutar la aplicación ASGI, Pydantic para validación interna, Transformers y Torch para cargar y ejecutar los modelos, PyYAML para leer la configuración, Jinja2 para renderizar la interfaz, Python Multipart para procesar formularios y archivos, LIME para explicabilidad y SentencePiece para compatibilidad con tokenizadores utilizados por Transformers.

4.7. Comprobación de dependencias instaladas

Una vez finalizada la instalación, puede comprobar que las dependencias principales se importan correctamente:

```
python -c "import fastapi, uvicorn, torch, transformers, yaml, jinja2, lime"
```

Si el comando termina sin mostrar errores, las librerías principales están disponibles en el entorno virtual.

5. Configuración del archivo config_app.yaml

La aplicación carga su configuración desde el archivo:

```
app/config_app.yaml
```

Este archivo contiene las rutas de los modelos y parámetros asociados a la inferencia. La configuración actual tiene la siguiente estructura:

```
modelo:
  ruta_modelo: "../models/binario/galen_1"
  ruta_tokenizer: "../models/binario/galen_1"
  max_length: 512

cascade:
  ruta_mama_model: "../models/mama/best_model"
  ruta_mama_labels: "../models/mama"
  ruta_resto_model: "../models/resto/best_model"
  ruta_resto_labels: "../models/resto"
  umbral_mama: 0.5
  umbral_resto: 0.2

api:
  titulo: "Clasificador de Neoplasias"
  version: "1.0"
```

Con esta configuración, la estructura esperada de la carpeta `models/` es:

```
models/
|-- binario/
|   |-- galen_1/
|-- mama/
|   |-- best_model/
|   |-- label2idx.json
|-- resto/
|   |-- best_model/
|   |-- label2idx.json
```

5.1. Descarga y colocación de los modelos

Los modelos entrenados no se incluyen en el repositorio porque ocupan demasiado espacio. Para ejecutar la aplicación, deben descargarse previamente desde el enlace de Drive facilitado junto con el proyecto.

Enlace de descarga de los modelos:

```
https://drive.google.com/file/d/1wLKNNolqbJa934kWXcT2a4e1VP2GGELT/view?usp=sharing
```

Una vez descargados, descomprima o copie los modelos dentro de la carpeta `models/` respetando la estructura indicada anteriormente. Es importante mantener los nombres de carpeta esperados por `app/config_app.yaml`, ya que la aplicación carga los modelos desde esas rutas relativas.

Si se cambia la ubicación o el nombre de alguna carpeta, será necesario actualizar las rutas correspondientes en `app/config_app.yaml`.

5.2. Sección modelo

La sección `modelo` corresponde al clasificador binario:

- `ruta_modelo`: ruta al directorio que contiene el modelo binario entrenado.
- `ruta_tokenizer`: ruta al tokenizador asociado al modelo binario.
- `max_length`: longitud máxima utilizada para tokenizar los textos de entrada.

El valor de `max_length` determina el tamaño máximo de secuencia empleado por el tokenizador antes de pasar el texto al modelo.

5.3. Sección cascade

La sección `cascade` configura la clasificación multietiqueta:

- `ruta_mama_model`: ruta al modelo multietiqueta especializado en etiquetas del grupo mama.
- `ruta_mama_labels`: ruta al directorio que contiene el archivo `label2idx.json` asociado al modelo mama.
- `ruta_resto_model`: ruta al modelo multietiqueta para el resto de localizaciones.
- `ruta_resto_labels`: ruta al directorio que contiene el archivo `label2idx.json` asociado al modelo resto.
- `umbral_mama`: umbral de decisión aplicado a las probabilidades del modelo mama.
- `umbral_resto`: umbral de decisión aplicado a las probabilidades del modelo resto.

Los modelos multietiqueta devuelven probabilidades independientes por etiqueta. Las etiquetas cuya probabilidad supera el umbral configurado se muestran como predicciones activas.

5.4. Sección api

La sección `api` contiene metadatos de la aplicación:

- `titulo`: nombre descriptivo de la aplicación.
- `version`: versión funcional del sistema.

6. Ejecución de la aplicación mediante Uvicorn

La aplicación debe ejecutarse desde la carpeta `app/`, ya que el archivo `main.py` carga `config_app.yaml`, `templates/` y `static/` mediante rutas relativas al directorio de trabajo.

Desde la raíz del proyecto, acceda a la carpeta de la aplicación:

```
cd app
```

Con el entorno virtual activado, ejecute el servidor:

```
python -m uvicorn main:app --reload
```

El parámetro `main:app` indica a Uvicorn que debe cargar la variable `app` definida en el archivo `main.py`. El parámetro `--reload` permite reiniciar automáticamente el servidor cuando se detectan cambios en el código fuente, lo que resulta útil durante la fase de desarrollo o demostración.

La salida esperada en terminal tendrá una forma similar a:

```
Uvicorn running on http://127.0.0.1:8000
```

Si se desea ejecutar la aplicación sin recarga automática, puede utilizarse:

```
python -m uvicorn main:app
```

7. Acceso a la interfaz web

Una vez iniciado el servidor, abra un navegador web e introduzca la siguiente dirección:

```
http://127.0.0.1:8000
```

La interfaz permite utilizar la aplicación de dos formas:

- Introduciendo directamente el texto clínico en el área de texto.
- Subiendo un archivo `.txt` codificado en UTF-8.

En ambos casos, el usuario puede seleccionar el modo de predicción antes de ejecutar el análisis.

8. Modos de análisis disponibles

8.1. Modo binario

El modo binario clasifica el texto clínico en una de las dos categorías principales:

- Una neoplasia
- Múltiples neoplasias

El flujo de uso es el siguiente:

1. Introducir o cargar una historia clínica en formato texto.
2. Seleccionar el modo **Binario**.
3. Opcionalmente, activar la explicación mediante LIME.
4. Pulsar el botón de análisis.
5. Revisar la clase predicha y la probabilidad asociada.

El resultado mostrado por la interfaz incluye la clase seleccionada por el modelo y un valor porcentual de confianza estimada para dicha clase.

8.2. Modo multietiqueta

El modo multietiqueta activa la inferencia en cascada configurada en `config_app.yaml`. Este modo está orientado a identificar posibles localizaciones tumorales a partir del texto clínico.

El flujo de uso es el siguiente:

1. Introducir o cargar una historia clínica en formato texto.
2. Seleccionar el modo **Multietiqueta**.
3. Pulsar el botón de análisis.
4. Revisar las etiquetas detectadas y las probabilidades asociadas.

En este modo se ejecutan los modelos configurados en la sección **cascade**. El sistema aplica los umbrales definidos para cada grupo y muestra las etiquetas cuya probabilidad supera el valor correspondiente.

9. Uso de explicaciones LIME

La aplicación incorpora explicaciones mediante LIME para el modo binario. Esta funcionalidad permite obtener una lista de fragmentos o palabras del texto que han contribuido a la decisión del modelo.

Para utilizar esta opción:

1. Seleccione el modo **Binario**.
2. Active la opción **Generar explicación con LIME**.
3. Introduzca o cargue el texto clínico.
4. Ejecute el análisis.

La interfaz mostrará una representación de las contribuciones de LIME, indicando el peso asociado a cada término relevante. Los pesos positivos y negativos permiten interpretar la dirección de la contribución respecto a la clase explicada.

Internamente, la función de explicabilidad se encuentra en:

```
app/lime_utils.py
```

El procedimiento utiliza **LimeTextExplainer** para generar perturbaciones del texto de entrada, consultar repetidamente al modelo y estimar la importancia local de los términos para la predicción obtenida.

10. Buenas prácticas de utilización

Para obtener un funcionamiento ordenado y reproducible se recomienda seguir las siguientes pautas:

- Ejecutar siempre la aplicación desde un entorno virtual dedicado al proyecto.
 - Mantener actualizado el archivo `app/requirements.txt` cuando se incorporen nuevas dependencias a la aplicación.
 - Verificar que las rutas de `app/config_app.yaml` apuntan a los directorios reales donde se encuentran los modelos.
 - Conservar la asociación entre cada modelo y su tokenizador correspondiente.
 - Utilizar textos clínicos anonimizados antes de introducirlos en la aplicación.
 - Emplear archivos `.txt` codificados en UTF-8 cuando se use la carga de documentos.
 - Ejecutar Uvicorn desde la carpeta `app/` con `python -m uvicorn main:app --reload`.
 - Revisar los umbrales de la sección `cascade` cuando se utilice el modo multitiqueta.
-

11. Comandos útiles de mantenimiento

11.1. Activar el entorno virtual

```
cd "C:\TFG\NeoplasiaClassifier-ES"  
.\.venv\Scripts\Activate.ps1
```

11.2. Reinstalar dependencias

```
python -m pip install -r app\requirements.txt
```

11.3. Mostrar paquetes instalados

```
python -m pip list
```

11.4. Ejecutar la aplicación

```
cd "C:\TFG\NeoplasiaClassifier-ES\app"  
python -m uvicorn main:app --reload
```

11.5. Detener la aplicación

Para detener el servidor, vuelva a la terminal donde se está ejecutando Uvicorn y pulse:

Ctrl + C
